

Capítulo 8

Circuitos Decodificadores

Decodificadores são sistemas digitais que executam a conversão ou tradução de um código binário para outro. Codificadores são os circuitos inversos, ou seja, também convertem de um código para outro. A denominação depende do ponto de referência ou seja, qual código deseja-se tomar como base para conversão. A figura 8.1 esquematiza o processo de (de)codificação.

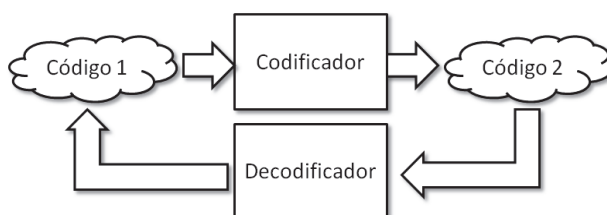


Figura 8.1: Modelo esquemático do problema de codificação/decodificação de códigos binários.

Formalmente, o processo de codificação pode ser descrito como na equação 8.1 onde f é a função de codificação, X é uma palavra no código original, Y é a palavra resultante do processo de decodificação e m, n referem-se ao comprimento das palavras nos códigos X e Y respectivamente.

$$f : X \mapsto Y, \quad X \in \mathbb{B}^m \wedge Y \in \mathbb{B}^n \quad (8.1)$$

A equação 8.2 descreve o processo inverso. a função de decodificação g toma como entrada um código Y e o decodifica para X .

$$g : Y \mapsto X, \quad X \in \mathbb{B}^m \wedge Y \in \mathbb{B}^n \quad (8.2)$$

Neste capítulo serão apresentados circuitos que executam a conversão entre os diversos códigos apresentados no capítulo 7.

8.1 Decodificador BCD8421 - binário

Decodificar um código em BCD8421 para binário pode ser muito simples ou muito complicado, depende do número de nibbles usados pelo código BCD para representar uma dada quantidade. No caso mais simples, apenas um nibble ou seja, 4 bits são utilizados para representar uma quantidade em BCD. A tabela 8.1 lista todas as possíveis entradas para o código BCD8421 com palavras de 4 bits.

Decimal	Binário	BCD8421	Decimal	Binário	BCD8421
0	0000	0000	8	1000	1000
1	0001	0001	9	1001	1001
2	0010	0010	10	1010	E
3	0011	0011	11	1011	E
4	0100	0100	12	1100	E
5	0101	0101	13	1101	E
6	0110	0110	14	1110	E
7	0111	0111	15	1111	E

Tabela 8.1: Tabela de correspondência entre decimal e binário e BCD8421.

Note que para representações maiores que 9 o código BCD8421 não é definido e conseqüentemente significa que algum erro ocorreu. Para os casos na faixa $[0, 9]$ a representação em binário é exatamente a mesma que a usada pelo código BCD8421. Basta então que se crie um circuito que informe quando um código inválido ocorra. Observando a tabela 8.1 observa-se que os 6 últimos casos são os que devem ser capturados para geração do sinal de erro. Tal fato pode ser codificado usando o mapa de Karnaugh apresentado na figura 8.2. Associando-se $A \rightarrow bcd3$, $B \rightarrow bcd2$, $C \rightarrow bcd1$ e $D \rightarrow bcd0$, obtêm-se a expressão mínima $Err_{min} = (bcd3 \cdot bcd1) + (bcd3 \cdot bcd2)$.

O circuito equivalente é apresentado na figura 8.3. Note que o circuito apresentado pode fornecer como saída o código BCD inválido.

No entanto o sinal de erro será habilitado. Fica a cargo de outro subsistema que utiliza o circuito descrito ignorar o código inválido com base no sinal de erro.

A situação fica muito mais complicada quando consideramos quantidades representadas em BCD superiores a 9. Em princípio seria possível definir uma tabela verdade que listasse sistematicamente todos os casos. Considere por exemplo o caso imediatamente superior, ou seja, 8 bits ou dois nibbles BCD. Neste caso 8 bits seriam a entrada em BCD resultando em $99_{10} = 1100011_2$ como valor máximo representável o que requer 7 bits para representação em binário puro. Neste caso a tabela verdade possuiria 256 linhas para a listagem

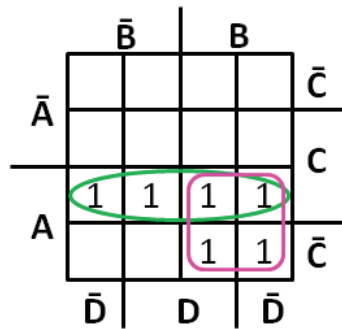


Figura 8.2: Mapa de Karnaugh para os 6 casos em que a conversão de BCD para binário geram erro.

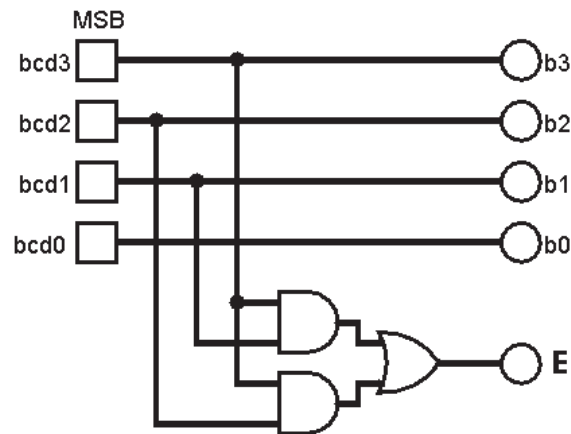


Figura 8.3: Circuito com detecção de código inválido para a conversão de BCD8421 para binário.

sistemática (incluindo casos inválidos) e 7 saídas, o que resultaria em mapas de Karnaugh de 8 variáveis, basicamente impraticável. Outra abordagem seria implementar um circuito capaz de somar (visto em detalhes no capítulo 9) dois números. A ideia básica reside em proceder a conversão direta dos 4 bits menos significativos e então converter os quatro bits mais significativos para dezenas. A conversão seria $0000 \rightarrow 0000000$, $0001 \rightarrow 0001010$, $0010 \rightarrow 0010100$, $0011 \rightarrow 0011110$, $0100 \rightarrow 0101000$, $0101 \rightarrow 0110010$, $0110 \rightarrow 0111100$, $0111 \rightarrow 1000110$, $1000 \rightarrow 1010000$, $1001 \rightarrow 1011010$. Em princípio seria possível definir um circuito decodificador para o nibble mais significativo. Outra abordagem mais simples refere-se a utilização de uma memória de apenas leitura (ROMs e outras memórias serão abordadas no capítulo XXXX). O circuito apresentado na figura 8.4 implementa o esquema descrito

acima.

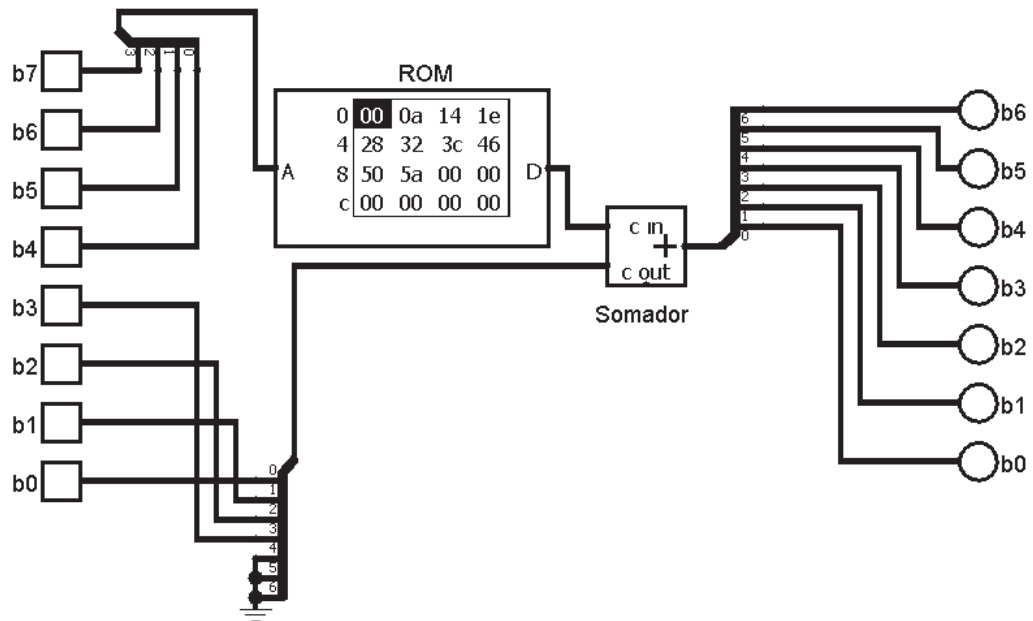


Figura 8.4: Circuito para conversão de BCD8421 de dois dígitos para binário.

O circuito apresentado na figura 8.4 não implementa controle de erros para os casos não revistos pelo código BCD8421. O mesmo esquema de controle desenvolvido no circuito da figura 8.3 pode ser empregado para geração do sinal de Erro.

Finalmente, a extensão empregada para dois nibbles por ser estendida para um número arbitrariamente grande. Basta que a memória ROM acomode os valores em binário das centenas, milhares, etc e que os somadores adicionais sejam suficientemente grandes.

8.2 Decodificador BCD8421 - 9876543210

A decodificação do código BCD8421 para o código 9876543210 é consideravelmente mais simples se comparada ao exemplo anterior. Como visto no capítulo 7 o código em questão gera um "1" em apenas uma das 10 saídas e "0" em todas as demais dependendo do número em BCD fornecido como entrada. Para implementar seu circuito equivalente deve-se simplificar cada uma das 10 saídas usando mapas de Karnaugh de 4 variáveis. Lançando mão dos seis estados irrelevantes (que compreendem as entradas na faixa [1010, 1111]) obtêm-se as seguintes expressões mínimas.

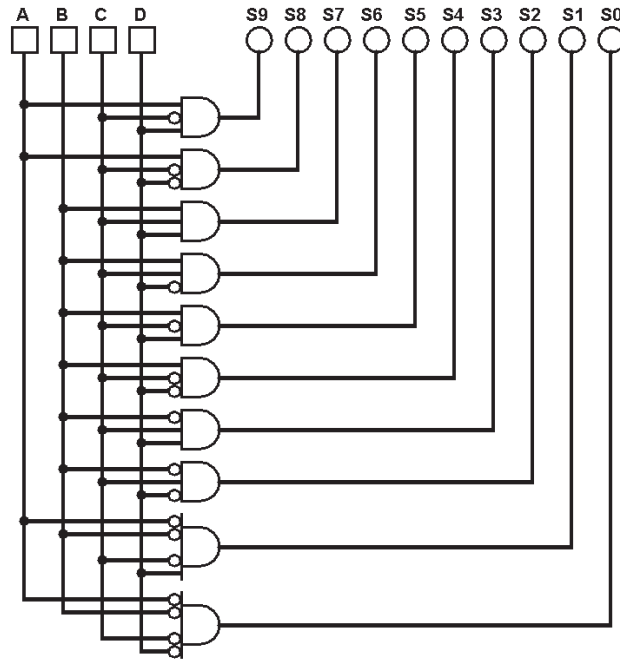


Figura 8.5: Diagrama de portas lógicas do conversor de BCD8421 para o código 9876543210.

$$\begin{aligned}
 S_1 &= \overline{A} \cdot \overline{B} \cdot \overline{C} \cdot D \\
 S_2 &= \overline{B} \cdot C \cdot \overline{D} \\
 S_3 &= \overline{B} \cdot C \cdot D \\
 S_4 &= B \cdot \overline{C} \cdot \overline{D} \\
 S_5 &= B \cdot \overline{C} \cdot D \\
 S_6 &= B \cdot C \cdot \overline{D} \\
 S_7 &= B \cdot C \cdot D \\
 S_8 &= A \cdot \overline{C} \cdot \overline{D} \\
 S_9 &= A \cdot \overline{C} \cdot D \\
 S_0 &= \overline{A} \cdot \overline{B} \cdot \overline{C} \cdot \overline{D}
 \end{aligned}$$

A implementação direta das expressões obtidas resulta no diagrama de portas lógicas apresentado na figura 8.5.

A decodificação inversa, ou seja, do código 9876543210 para o código BCD8421 se dá de forma similar. No entanto neste caso tem-se 10 entradas e 4 saídas. Utilizar o mapa de Karnaugh nesta situação pode ser problemático pois a tabela verdade terá 1024 linhas. Uma abordagem alternativa refere-se a utilizar um contador de bits seguido de um comparador para detectar possíveis erros no código. Note que qualquer combinação de entradas no

código 98786543210 em que mais de um bit seja "1" se configura como um erro no código.

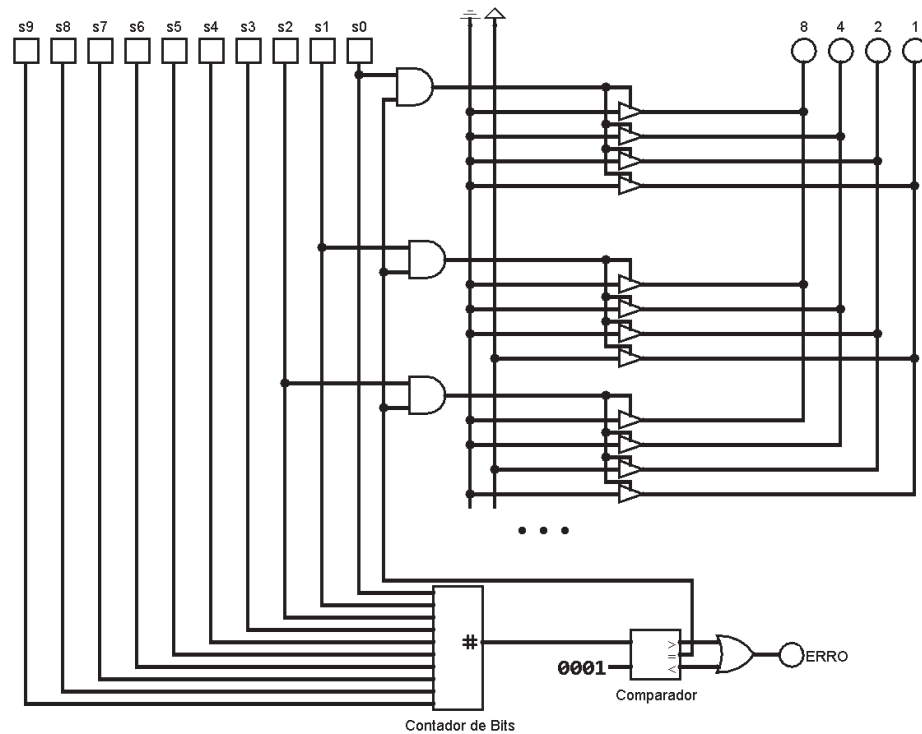


Figura 8.6: Diagrama de portas lógicas do conversor 9876543210 para BCD8421.

A figura 8.6 apresenta o diagrama de portas lógicas do decodificador. Além do detector de erros no código implementado usando o contador de bits e o comparador o circuito produz a saída em BCD usando para tal 4 buffers tri-state para cada uma das dez entradas. A figura apresenta apenas as ligações para os três primeiros casos.

8.3 Decodificador BCD8421 - Excesso de 3

Não há nada de especial nos circuitos decodificadores tanto de BCD para Excesso de 3 e vice versa. Ambos os casos possuem quatro bits de entrada e quatro de saída. Sendo assim, a aplicação do método de mapas de Karnaugh é suficientemente simples para a obtenção do circuitos codificador e decodificador.

As expressões mínimas obtidas a partir dos quatro mapas de Karnaugh

são apresentadas na equação 8.3 e o circuito equivalente é mostrado pela figura 8.7.

$$\begin{aligned}
 S_3 &= A + BD + BC \\
 S_2 &= \overline{B}D + \overline{B}C + B\overline{C} \cdot \overline{D} \\
 S_1 &= C \odot D \\
 S_0 &= \overline{D}
 \end{aligned}
 \tag{8.3}$$

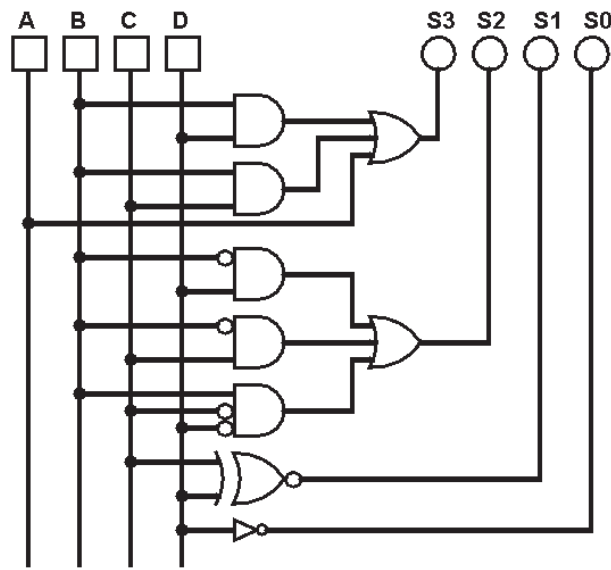


Figura 8.7: Circuito decodificador do código BCD8421 para Excesso de 3.

O processo inverso, ou seja a conversão de Excesso de 3 para BCD8421 segue o mesmo processo. Após a seleção dos mintermos, transposição para os mapas de Karnaugh e simplificação obter-se-ão as expressões apresentadas na equação 8.4.

$$\begin{aligned}
 S_8 &= AB + ACD \\
 S_4 &= \overline{B} \cdot \overline{D} + A\overline{C}D + BCD \\
 S_2 &= C \oplus D \\
 S_1 &= \overline{D}
 \end{aligned}
 \tag{8.4}$$

A transcrição das expressões para o diagrama de portas lógicas resulta no circuito apresentado pela figura 8.8.

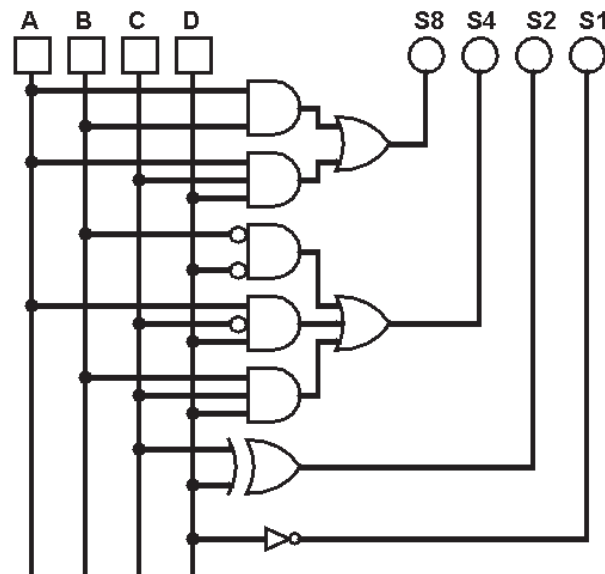


Figura 8.8: Circuito decodificador do código Excesso de 3 para BCD8421.

8.4 Decodificador BCD8421 - Gray

Como visto no capítulo 7 o código Gray é aquele que códigos sucessivos diferem em apenas um bit. A aplicação direta da equação 7.1 para $n = 4$ resulta no simples circuito apresentado pela figura 8.9.

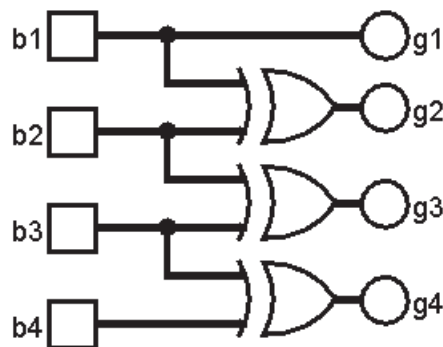


Figura 8.9: Circuito codificador do código BCD8421 para o código Gray.

A decodificação, ou seja, conversão de Gray para binário pode ser obtida pela equação 7.2 resultando no circuito apresentado pela figura 8.10.

Um fato interessante acerca dos circuitos codificadores e decodificadores entre os códigos Gray e binário é o fato de que sua realização seja tão simples

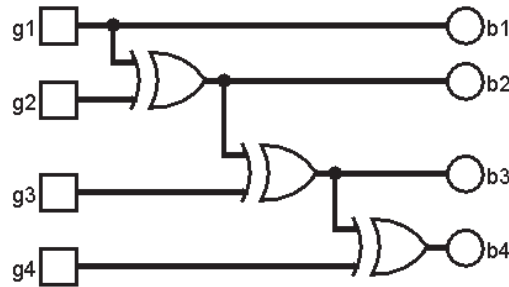


Figura 8.10: Circuito codificador do código Gray para o código BCD8421.

frente a complexidade do código em si. Este é um dos fatores que motivam o código Gray ser tão utilizado em diversas áreas.

8.5 Decodificador BCD8421 - Johnson

O código de Johnson é circular e geralmente construído via uma cadeia circular de registradores de deslocamento. No entanto ele também pode possuir uma realização combinacional simples de ser obtida visto que o comprimento da palavra de entrada (4 bits) é gerenciável via manipulação algébrica ou por meio de mapas de Karnaugh. As expressões mínimas para as cinco saídas do código de Johnson são apresentadas a seguir:

$$\begin{aligned}
 S_4 &= A + BD + BC \\
 S_3 &= B + A\bar{D} \\
 S_2 &= B + CD \\
 S_1 &= B\bar{C} + \bar{B}C + C\bar{D} \\
 S_0 &= \bar{A}\bar{C}D + B\bar{C} + \bar{B}C
 \end{aligned}$$

As equações mínimas podem ser realizadas pela implementação das equações mínimas descritas como apresentado na figura 8.11.

A implementação do decodificador, ou seja, da conversão de Johnson para binário pode ser feita seguindo as mesmas linhas descritas anteriormente. No entanto, considere o código de johnson para 10 bits de saída, ou seja, que começa com o código $0_{10} \rightarrow 00000\ 00000_J$ e termina com o código $19_{10} \rightarrow 10000\ 00000_J$. Tal código teria 8 bits de entrada pois são necessários 2 nibbles para representar 19_{10} em BCD8421 e naturalmente 10 bits de saída. Claramente a abordagem combinacional seria impraticável. Uma solução no entanto simples para o problema descrito será apresentada no capítulo XXXX.

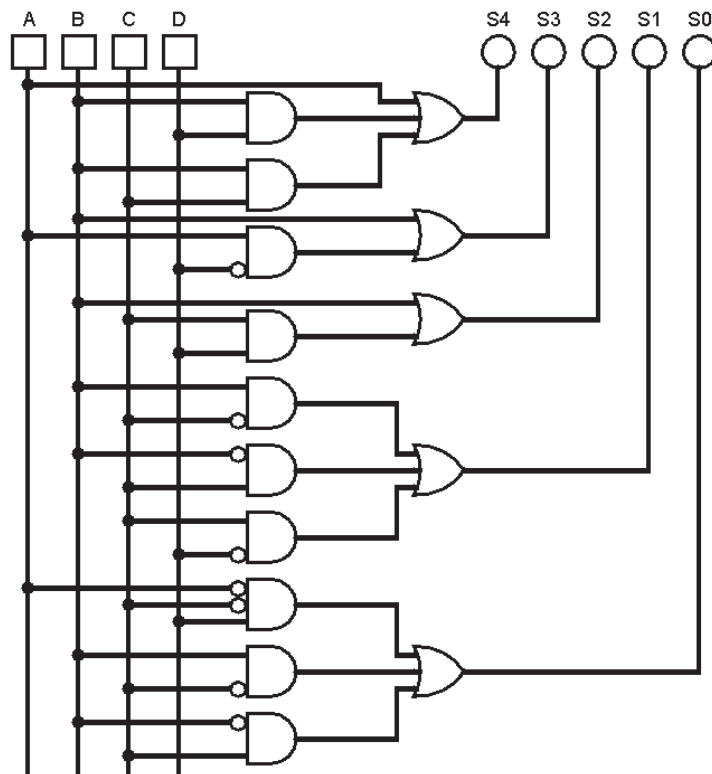


Figura 8.11: Circuito mínimo para o codificador de BCD para o código de Johnson.

8.6 Decodificador para 7 Segmentos

Um código binário muito utilizado na prática e não coberto no capítulo 7 refere-se ao código de acionamento de displays de sete segmentos. Estes são dispositivos de saída de dados utilizados em diversos dispositivos como balanças, calculadoras, etc. A figura 8.12 apresenta três versões esquemática e a direita uma foto de um típico display de sete segmentos.

Fundamentalmente, um display de sete segmentos nada mais é que um conjunto de 7 LEDs dispostos de maneira fixa a formar um 8. Os LEDs podem ser acionados individualmente permitindo que quantidades em decimal, binário Octal ou até mesmo Hexadecimal sejam apresentados. Em sua realização mais comum eles podem ser o que se convencionou chamar de anodo comum ou catodo comum o que corresponde a dizer que os LEDs são acionados via nível lógico "0" e "1" respectivamente.

Hoje em dia a aritmética digital é quase que na totalidade executada em binário. No entanto a apresentação de resultados em binário é pouco

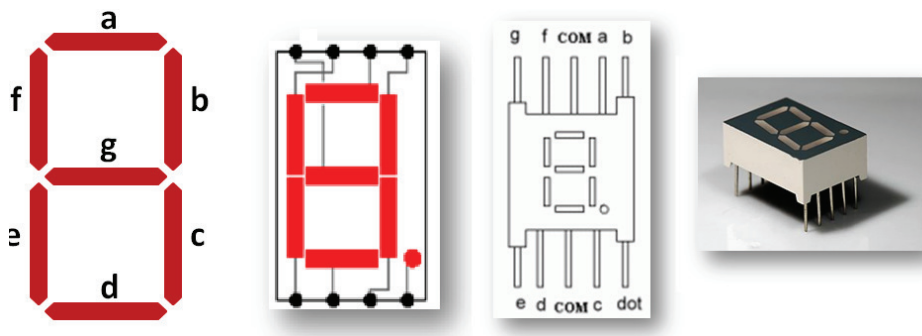


Figura 8.12: Quatro representações do display de sete segmentos. Da esquerda para a direita: a) Nomeação usando letras para cada um dos segmentos; b) ligação usual dos LEDs com os terminadores do componente; c) esquema do componente apresentando dois terminais comuns (com = terra) e finalmente d) uma foto do componente real.

representativa do ponto de vista da pessoa comum. Consequentemente para apresentação de informação utiliza-se algum tipo de mostrador que em geral apresenta os dados em decimal. Considere por exemplo uma balança de verduras que apresentasse o peso de um saco de laranjas para uma dona de casa acendendo uma sequência de LEDs acionados pelo número binário 101011011g para representar 347g. Claramente esta não é a abordagem mais adequada.

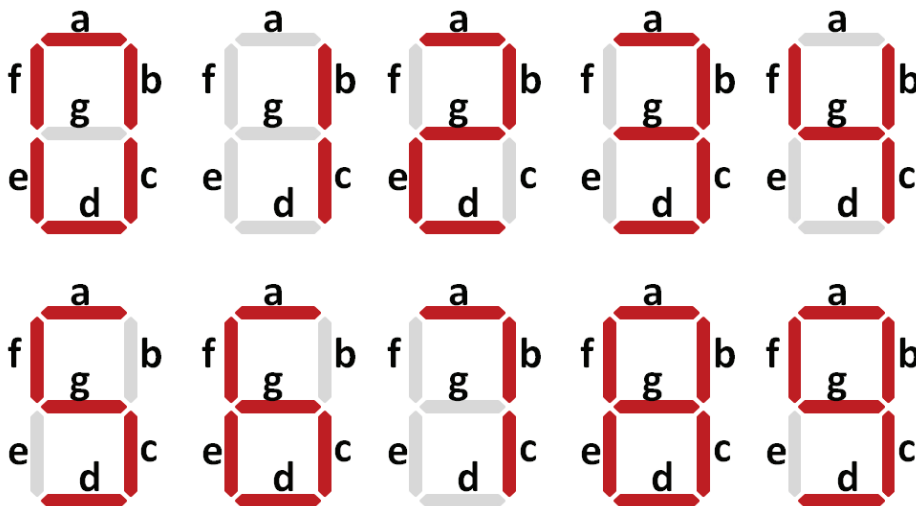


Figura 8.13: Padrões de ativação dos LEDs do display de 7 segmentos para representação dos dígitos decimais.

O código de 7 segmentos fornece uma alternativa simples para implementação de circuitos que precisam fornecer dados numéricos de saída para o usuário. Observando a figura 8.13 fica claro que os dígitos decimais podem ser "desenhados" via padrões de ativação dos 7 LEDs distintos. Outra forma de entender o padrão de ativação seria listando os LEDs que devem ser acessos ou seja, $0 \rightarrow abcdef$, $1 \rightarrow bc$, $2 \rightarrow abdg$, $3 \rightarrow abcdg$, $4 \rightarrow bcfg$, $5 \rightarrow acdfg$, $6 \rightarrow acdefg$, $7 \rightarrow abc$, $8 \rightarrow abcdefg$ e $9 \rightarrow abcdfg$. Este padrão é capturado de maneira sistemática na tabela ??.

decimal	A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	0	1	1	1	1	1	1	0
1	0	0	0	1	0	1	1	0	0	0	0
2	0	0	1	0	1	1	0	1	1	0	1
3	0	0	1	1	1	1	1	1	0	0	1
4	0	1	0	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1	0	1	1
6	0	1	1	0	1	0	1	1	1	1	1
7	0	1	1	1	1	1	1	0	0	0	0
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	1	0	1	1
10	1	0	1	0	-	-	-	-	-	-	-
11	1	0	1	1	-	-	-	-	-	-	-
12	1	1	0	0	-	-	-	-	-	-	-
13	1	1	0	1	-	-	-	-	-	-	-
14	1	1	1	0	-	-	-	-	-	-	-
15	1	1	1	1	-	-	-	-	-	-	-

Tabela 8.2: Tabela de equivalência entre os códigos decimal, BCD8421 e 7 segmentos.

A partir da tabela 8.6 pode-se extrair as sete funções de saída a-g resultando em:

$$a = A + C + B \odot D$$

$$b = B + C \odot D$$

$$c = \overline{C} + B + D$$

$$d = A + \overline{B}.\overline{D} + \overline{B}C + C\overline{D} + B\overline{C}D$$

$$e = \overline{B}.\overline{D} + C\overline{D}$$

$$f = A + \overline{C}.\overline{D} + B\overline{C} + B\overline{D}$$

$$g = A + C\overline{D} + B \oplus C$$

que podem ser implementadas resultando no circuito apresentado na figura 8.14. Note que qualquer entrada fora da faixa [0,9] resultará em um padrão de acionamento errado no display de 7 segmentos.

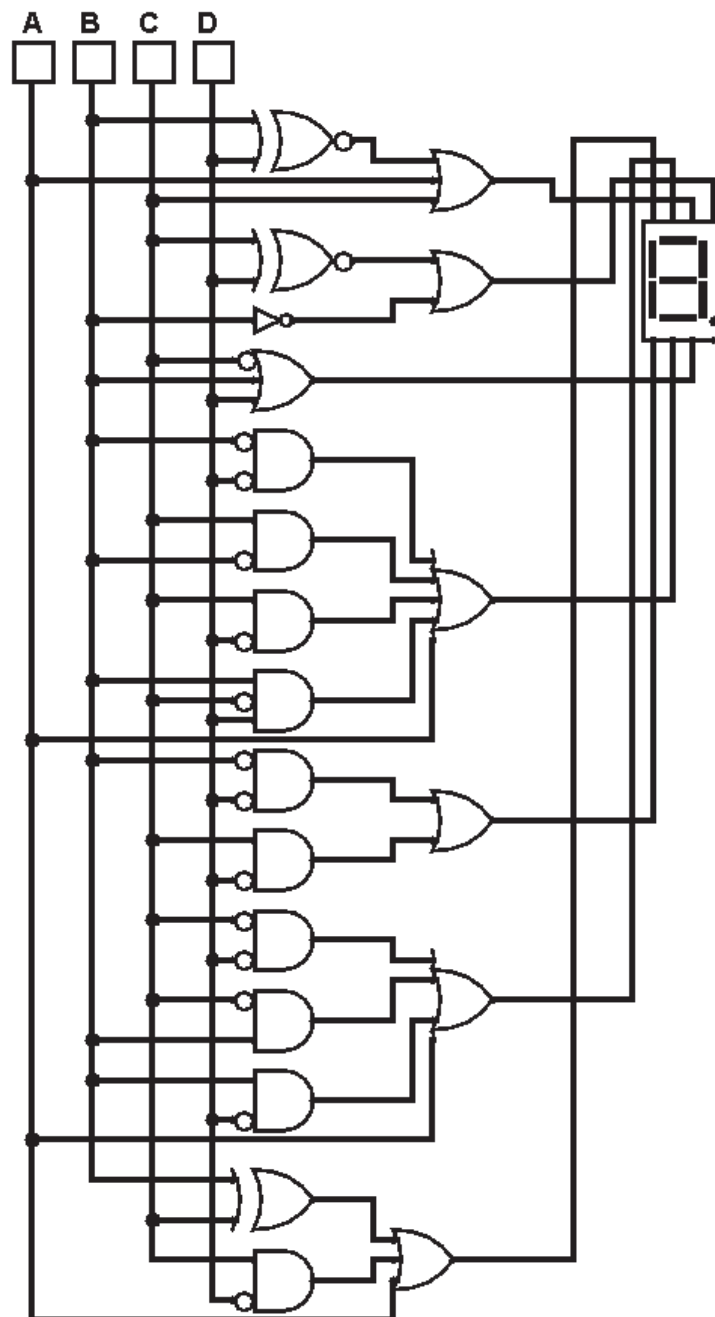


Figura 8.14: Circuito decodificador de BCD8421 para 7 segmentos.

Por fim observe que não há nada que impeça que outros padrões de ativação sejam alcançados como exemplificado na figura [8.15](#).

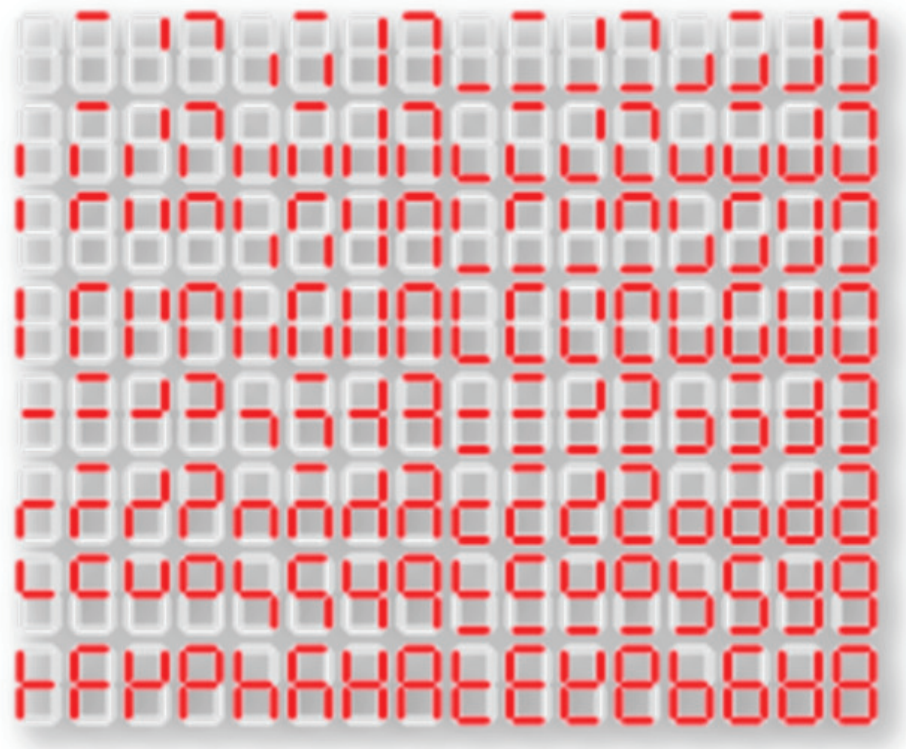


Figura 8.15: Exemplo de outros padrões de ativação dos 7 LEDs do display de 7 segmentos.

Exercícios

1. Implemente o circuito de detecção de erros no código BCD para o circuito apresentado na figura 8.4.
2. Construa um diagrama esquemático do circuito necessário para decodificar um número em binário para o código BCD8421. O circuito deve testar se o número é maior que 99 e em caso afirmativo fornecer um erro. Caso contrário proceda com a conversão que resultará em um número de dois nibbles, o mais significativo para a dezena e o menos significativo para a unidade.
3. Construa os dez mapas de Karnaugh com situações irrelevantes necessários para a implementação do decodificador $BCD8421 \rightarrow 9876543210$. A seguir simplifique os mapas e verifique se as saídas mínimas correspondem a:

$$S_1 = \overline{A} \cdot \overline{B} \cdot \overline{C} \cdot D$$

$$S_2 = \overline{B} \cdot C \cdot \overline{D}$$

$$S_3 = \overline{B} \cdot C \cdot D$$

$$S_4 = B \cdot \overline{C} \cdot \overline{D}$$

$$S_5 = B \cdot \overline{C} \cdot D$$

$$S_6 = B \cdot C \cdot \overline{D}$$

$$S_7 = B \cdot C \cdot D$$

$$S_8 = A \cdot \overline{C} \cdot \overline{D}$$

$$S_9 = A \cdot \overline{C} \cdot D$$

$$S_0 = \overline{A} \cdot \overline{B} \cdot \overline{C} \cdot \overline{D}$$

4. Construa um sistema digital que teste se dado um código em 9876543210 ele equivale a um código válido ou se ele contém algum erro. Note que um código válido no referido sistema de codificação implica que um e apenas um bit dentre os 10 que compõem o código é "1", e os nove remanescentes "0".
5. Complete o circuito parcial apresentado na figura 8.6 para os sete casos faltantes.
6. Implemente o circuito de detecção de erros para o conversor de BCD8421 para Excesso de 3 para complementar o circuito apresentado na figura 8.7.
7. Com base na tabela verdade de codificação entre BCD8421 para Excesso de 3 construa os quatro mapas de Karnaugh e verifique que as saídas S_3 , S_2 , S_1 e S_0 produzem as saídas mínimas.

$$\begin{aligned}
S_3 &= A + BD + BC \\
S_2 &= \overline{B}D + \overline{B}C + \overline{B}C \cdot \overline{D} \\
S_1 &= C \odot D \\
S_0 &= \overline{D}
\end{aligned}$$

8. Com base na tabela verdade de codificação entre Excesso de 3 para BCD8421 construa os quatro mapas de Karnaugh e verifique que as saídas S_8 , S_4 , S_3 e S_1 produzem as saídas mínimas.

$$\begin{aligned}
S_8 &= AB + ACD \\
S_4 &= \overline{B} \cdot \overline{D} + \overline{A}CD + BCD \\
S_2 &= C \oplus D \\
S_1 &= \overline{D}
\end{aligned}$$

9. Utilizando a equação 7.1 construa um circuito codificador de binário para Gray onde a palavra de entrada tenha comprimento de 8 bits.
10. Utilizando a equação 7.2 construa um circuito codificador de Gray para binário onde a palavra de entrada tenha comprimento de 16 bits.
11. Construa a tabela de correspondência entre códigos para o código de Johnson para o BCD8421 (não esqueça de listar os casos irrelevantes). A seguir levante as quatro funções canônicas de saída e simplifique-as via manipulação algébrica ou por meio de mapas de karnaugh. Por fim implemente o diagrama de portas lógicas.
12. Seria possível utilizar o display de 7 segmentos para apresentar caracteres alfanuméricos? Desenhe as representações em sete segmentos do alfabeto adotado na língua portuguesa. A seguir monte a tabela verdade que correlaciona o código em ASCII de cada uma das letras e números da tabela ASCII com o padrão de acionamento de LEDS no código 7 segmentos. Por fim simplifique e implemente o circuito.